Förslag: säkrare och skarpare skanning

Kodändring i barcode_scanner.py, utifrån mätningarna 17–20 oktober 2026

Status: **förslag, inte genomfört.** Inget ändras förrän Elias godkänt. Filen ändras på en gren, testas och läggs på main via en pull request. Varje ändring numreras i filhuvudet (ändring 20 och framåt), som tidigare.

1. Bakgrund

Mätningarna har visat fyra saker som skannern inte hanterar i dag:

Problem	Mätt	Följd i dag
Felaktigt djup de första sekunderna	3–13 m i stället för 0,20 m, i 5 körningar	Positionen blir 1–10 m fel
Felläsningar med giltig kontrollsiffra	62 av 1 471 läsningar (~4 %), nästan alla från en tunn remsa av koden	Fel vara kan sägas till användaren
Autofokus fokuserar på bildens mitt	Kod längst ner: 0 %. Kod i mitten: 100 %	Koden blir suddig och läses inte
Rätt linsläge varierar	Fönstret flyttas med avstånd och vinkel (69 / 77–92 / 70–77)	Ett fast värde fungerar inte

2. Översikt

Förslaget delas i två delar som kan godkännas var för sig. Del A är ren beräkning på det som redan loggas och har låg risk. Del B styr kameran och måste provas på Pi:n.

Del	Ändring	Risk	Kan testas utan Pi
A1	Djupfilter: avvisa djup som inte stämmer med kodens storlek	Låg	Ja (loggar)
A2	Höjdfilter: avvisa läsningar under 15 % höjd/bredd	Låg	Ja (loggar)
B1	Fokuslås vid träff: lås linsen där koden lästes, släpp efter 1,5 s utan träff	Medel	Nej
B2	Fokusområde på koden: hitta kodens plats och styra autofokus dit	Högre	Delvis (sparade bilder)

Rekommendation: genomför A1 och A2 först, sedan B1. B2 förbereds med ett test på sparade bilder innan något skrivs in i skannern.

3. Del A: filter (vad som räknas som en giltig läsning)

A1. Djupfilter

Djupet och kodens storlek i bilden hänger ihop: ju längre bort, desto färre pixlar per streck. Uppmätt för tomatpurén (modul ~0,35 mm): djup × px/modul ≈ 0,51–0,55 m i 1080p och ≈ 1,0 m i 4K. Andra varor har andra modulstorlekar (standarden tillåter ungefär 0,26–0,66 mm), så kontrollen görs med bred marginal:

```
förväntat_djup = K[läge] / px_per_modul    (K = 0,52 i 1080p, 1,0 i 4K)
avvisa om djup > 3 × förväntat_djup eller djup < förväntat_djup / 3
```

Med faktor 3 godkänns moduler från ~0,12 till ~1 mm, medan de felaktiga djupen (15–60 gånger för stora) stoppas. Läsningen behålls som avläsning, men positionen räknas inte ut och inget sägs till användaren.

A2. Höjdfilter

Kodens höjd i bilden jämförs med dess bredd. Höjden mäts längs kodens sidor (vänster och höger kant), så att det fungerar även när koden är roterad:

```
höjd = (avstånd(övre_vänster, nedre_vänster) + avstånd(övre_höger, nedre_höger)) / 2
avvisa om höjd / bredd < 0,15
```

Uppmätt på alla loggar med facit: stoppar 60 av 62 felläsningar och kastar 135 av 1 409 rätta (10 %). Gränsen är en konstant överst i filen, så att den kan justeras efter validering.

Gemensamt för A1 och A2

- **Allt loggas som förut.** Varje rad får ett nytt fält, filter: tomt, ”tunn” eller ”orimligt djup”, samt hojd_andel och forvantat_djup_m. Då kan filtren utvärderas i efterhand på samma sätt som i dag.
- Terminalen visar avvisade läsningar på en kort rad (t.ex. ”avvisad: tunn remsa”), inte som vara och position.
- Sammanfattningen vid Ctrl+C visar hur många läsningar varje filter stoppade, och med --facit hur många rätta och fel.

4. Del B: fokusstyrning

I dag ställs fokus bara en gång vid start. För B1 och B2 behöver skannern kunna skicka kommandon till kameran medan den kör. Pipelinen får därför en styrkanal (XLinkIn kopplad till kamerans inputControl). B är avstängt när --fokus används, så att fasta tester fungerar som förut.

B1. Fokuslås vid träff (Elias idé)

- När en läsning passerar filtren i del A låses linsen på det linsvärde bildrutan hade (setManualFocus).
- Så länge koden läses ligger låsningen kvar. Det ger många läsningar i rad, vilket röstningen behöver senare.
- Om ingen giltig läsning kommer på 1,5 s släpps låsningen och autofokus tar över igen.
- Varje lås och släpp loggas med linsvärde och tid.

B2. Fokusområde på koden

Autofokus ställer in sig på bildens mitt. Om skannern vet var koden sitter, även när den är suddig, kan autofokus styras dit (setAutoFocusRegion). Problemet är att zxing bara hittar koder som går att läsa. Förslaget är därför en separat detektor som hittar streckkodsliknande områden utan att avkoda dem, till exempel OpenCV:s BarcodeDetector.

Osäkerheter i B2 som måste kontrolleras först: (1) Att OpenCV-versionen på Pi:n har BarcodeDetector. (2) Att den hittar en suddig kod. Testas på sparade bilder där koden var suddig (t.ex. bild_nere.jpg) innan något skrivs in i skannern. (3) Vilka koordinater setAutoFocusRegion förväntar sig (sensor eller bild) på OAK-D Lite. (4) Hur mycket CPU detektorn kostar per bildruta.

5. Vad som inte ändras

- Kartan, positionsberäkningen, kommandoraden (--diag, --4k, --facit, --fokus) och loggformatet (bara nya fält läggs till).
- Röstning, produktlista, röstvägledning, knappskanning och detect_spatial.py ingår inte. De kommer senare.
- Skannern är fortfarande fristående och körs aldrig samtidigt som detect_spatial.py.

6. Testplan

Del	Test	Godkänt om
A	Kör filtren på alla gamla loggar	Samma siffror som analysen: 60 av 62 fel stoppas, 135 rätta kastas
A	Syntetiska koder (rak, roterad 90°, tunn remsa)	Höjd och px/modul blir rätt, tunn remsa avvisas
A	Nya körningar nära pixelgränsen (1080p ~0,23–0,25 m) med --facit	Inga felläsningar passerar, minst 80 % av rätta behålls
A	Låst lins (felaktigt djup i början)	Inga positioner med orimligt djup skrivs ut
B1	Kod längst ner i bilden (autofokus missar)	Efter första träffen läses koden stabilt
B1	Flytta paketet 2–3 cm under körning	Låsningen släpps och koden hittas igen
B2	Sparade suddiga bilder	Detektorn hittar kodens plats
B2	Kod längst ner, autofokus på	Fokus hamnar på koden och den läses

7. Beslut som behövs från Elias

- Godkänna del A (A1 och A2).
- Godkänna del B1, eller vänta tills A är validerad.
- Om B2 ska förberedas med ett test på sparade bilder (ingen ändring i skannern).

Parametrarna (faktor 3, 15 %, 1,5 s) är startvärden från mätningarna och ligger som konstanter överst i filen.